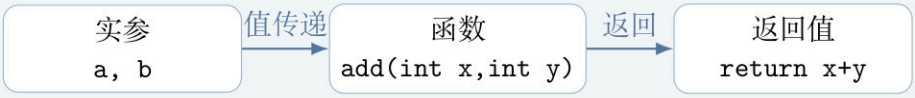
C++ 函数

定义 · 调用 · 值传递 · 声明 · 分文件编写

从“可复用代码块”到“编译—链接”的完整初学者链路



专题定位　C++ 程序设计基础 · 函数专题
适用阶段　第一次学习函数、刚开始接触分文件编写与链接错误的本科生
核心问题　函数为什么存在？形参与实参如何对应？值传递为什么“不改原变量”？声明、定义与链接到底是什么关系？

目录

1 阅读说明：这份讲义怎样使用 **4**

2 本专题在课程体系中的位置 **4**

3 学科小史：为什么程序设计逐渐离不开函数 **4**

4 从课本到现实：函数到底有什么用 **5**

5 原始资料重新编码：手写笔记的主线是什么 **5**

6 原图纠错：这几处必须改 **5**

7 核心概念一：函数是什么，为什么要引入它 **6**

7.1 一个最小可运行例子 6

8 核心概念二：函数定义的结构 **7**

8.1 五个要素逐项解释 7

9 核心概念三：形参与实参 **8**

10 核心概念四：函数调用与返回过程 **8**

11 核心概念五：值传递为什么不会改掉实参 **9**

12 核心概念六：四种基础函数形式 **9**

13 核心概念七：函数声明与函数定义 **10**

14 核心概念八：分文件编写到底发生了什么 **11**

14.1 最推荐的初学者三文件结构 11

14.2 为什么会出现 undefined reference 12

15 相近概念辨析 **12**

16 高频易错点 **12**

17 本专题知识地图 **13**

18 学完后的自检 **13**

19 知识点—题目覆盖表 **14**

20 A. 基础巩固训练：25 题 **14**

20.1 A1. 基础单项选择题（1–10） 14
20.2 A2. 基础多项选择题（11–15） 16
20.3 A3. 基础填空题（16–20） 16
20.4 A4. 基础解答题（21–25） 17

21 B. 中等偏下提升训练：10 题 17
21.1 B1. 单项选择题（26–28） 17
21.2 B2. 多项选择题（29–30） 18
21.3 B3. 填空题（31–33） 18
21.4 B4. 综合大题（34–35） 18

22 C. 跨学科迁移训练：3 题 19

23 全部答案速查 20

24 基础题详细解析 20
24.1 单项选择 1–10 20
24.2 多项选择 11–15 21
24.3 填空题 16–20 22
24.4 基础解答题 21–25 22

25 提升题详细解析 24
25.1 单项选择 26–28 24
25.2 多项选择 29–30 24
25.3 填空题 31–33 24
25.4 综合大题 34 25
25.5 综合大题 35 25

26 跨学科迁移题详细解析 25

27 基础解题步骤总结 26

28 后续课程连接 26

29 参考资料与权威核验 26

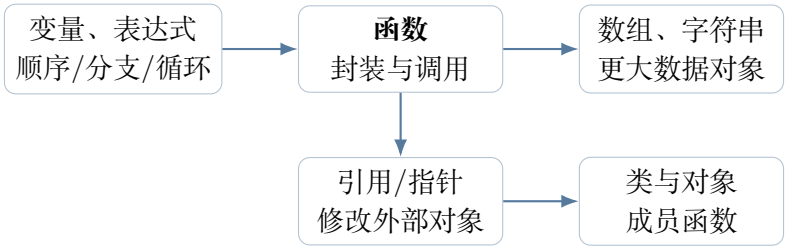
1 阅读说明：这份讲义怎样使用

这份资料不把你的手写笔记简单“抄成电子版”，而是把其中的主线重新组织为一条适合自学的知识链：**函数的作用 → 定义 → 调用 → 值传递 → 函数形式 → 声明 → 分文件编写 → 编译与链接**。你的提示词要求成品既能作为本科初学者讲义，又必须保留 LaTeX 源文件、实际用 XeLaTeX 编译并逐页检查；本讲义据此制作。

学完本专题，你应当能独立回答

1. 函数为什么能减少重复代码，并让程序结构更清楚？
2. 一个函数定义中，返回类型、函数名、形参列表、函数体、return 分别是什么？
3. 形参和实参到底有什么区别？
4. 为什么值传递时函数内部改了形参，外面的实参通常不变？
5. `void` 函数能不能写 `return 3;`？
6. 函数声明和函数定义分别解决编译过程中的什么问题？
7. 为什么“头文件写了声明”仍然可能出现 `undefined reference`？
8. 三文件工程中，`main.cpp`、`xxx.h`、`xxx.cpp` 应分别放什么？

2 本专题在课程体系中的位置

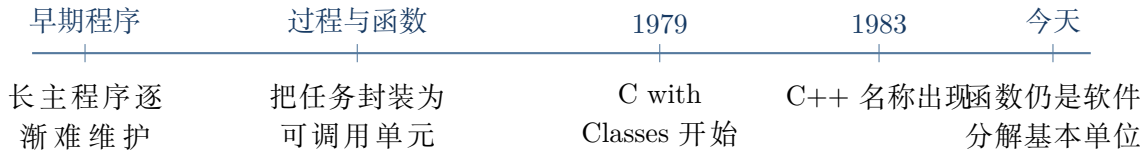


函数是从“会写一段代码”过渡到“会组织程序”的关键一步。后续无论是数组处理、数据结构、嵌入式程序、控制算法还是机器人程序，都会大量依赖函数来划分任务边界。

3 学科小史：为什么程序设计逐渐离不开函数

学科小史

函数思想并不是 C++ 才出现。早期程序规模不断扩大后，人们逐渐发现：如果把每个具体任务都直接写在一条长主程序里，代码会难以理解、难以复用，也难以多人协作。子程序、过程与函数因此成为结构化程序设计的重要工具。C++ 延续了 C 语言的函数机制，并在 20 世纪 80 年代逐步发展为支持更强类型检查、数据抽象与面向对象能力的语言。Bjarne Stroustrup 记载，他从 1979 年开始了后来成为 C++ 的工作，最初称为“C with Classes”，1983 年开始使用“C++”这一名称。



资料核验：历史核验主要参考 Bjarne Stroustrup 的 C++ FAQ 与《A History of C++: 1979–1991》。

4 从课本到现实：函数到底有什么用

从课本到现实

传感器换算。单片机读取 ADC 后，常写一个函数把原始数字量换算成电压、温度或光照度。
机器人控制。“读取编码器”、“计算速度”、“限制输出”、“发送串口数据”往往各自封装成函数。
工程计算。一段多次使用的单位换算、最大值判断、误差计算若封装成函数，修改时只改一个地方。
大型项目。不同.cpp 文件分别负责不同模块，函数声明构成模块之间的接口，链接器把它们最终组合成可执行程序。

5 原始资料重新编码：手写笔记的主线是什么

根据你上传的 5 页手写资料，可以可靠识别出的专题边界如下：

1. 函数概述：封装经常使用的代码，减少重复；较大的程序可按功能拆分。
2. 函数定义：返回值类型、函数名、参数列表、函数体、return 表达式。
3. 函数调用：使用已经定义或声明好的函数。
4. 值传递：实参把值传给形参，形参变化通常不影响实参。
5. 函数的四种基础形式：无参无返、有参无返、无参有返、有参有返。
6. 函数声明：告诉编译器函数名与调用接口；声明可重复，普通函数定义通常只保留一份。
7. 函数分文件编写：在.h 中写声明，在.cpp 中写定义，使用方通过 `#include` 获得声明。

个别手写标点、页背透印与很浅的零碎文字不影响上述核心主线，因此本讲义不对这些不可确认的局部字迹做“原句恢复”。

6 原图纠错：这几处必须改

纠错 1：void test_3 的返回类型与 return 不匹配

问题：返回类型是 `void`，却返回了一个 `int` 表达式。
规范写法：

```
int test_3(int a, int b) {
    return a + b;
}
```

如果函数真的不需要结果，则保留 `void`，只能使用 `return;`（或自然执行到函数末尾），不能用 `return a+b;`。

纠错 2：void test_4 的返回类型与 return 不匹配

同理，若希望返回 `temp`，应改成 `int test_4();`；若不返回值，就不应返回 `temp`。

纠错 3：最大值示例中的 result 必须先声明

原笔记中可见类似 `result = a > b ? a : b;`。若前面没有声明 `result`，应改为：

```
int result = (a > b) ? a : b;
return result;
```

或者直接写：

```
return (a > b) ? a : b;
```

纠错 4：“声明可多次，定义只一次”是初学者简化说法

对普通非 inline 函数，这句话作为入门规律基本可用：兼容声明可以重复出现，而程序需要一个可供链接的定义。更严格地说，C++ 受 One Definition Rule（ODR）约束；某些 inline / 模板情形允许在多个翻译单元出现等价定义。当前阶段记住“**声明可以重复，普通函数实现不要到处重复定义**”即可。

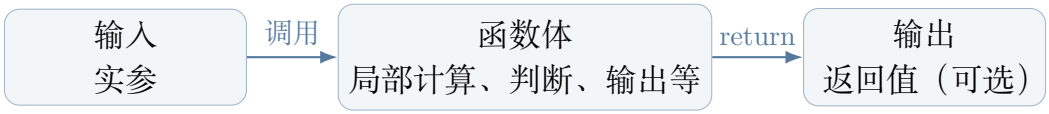
7 核心概念一：函数是什么，为什么要引入它

定义

函数（*function*）是一个具有名字、可以被调用的代码单元，用来完成一个相对明确的操作。它可以接收参数，也可以把结果返回给调用者。

直观理解

把函数想成一个“黑箱”：外部只需要知道**给它什么输入、它能完成什么功能、会给出什么输出**，不必每次都把内部实现重新写一遍。



7.1 一个最小可运行例子

```
#include <iostream>

int add(int num1, int num2) {
    int sum = num1 + num2;
    return sum;
}
```

```
int main() {
    int a = 3;
    int b = 5;
    int result = add(a, b);
    std::cout << result << '\n';
    return 0;
}
```

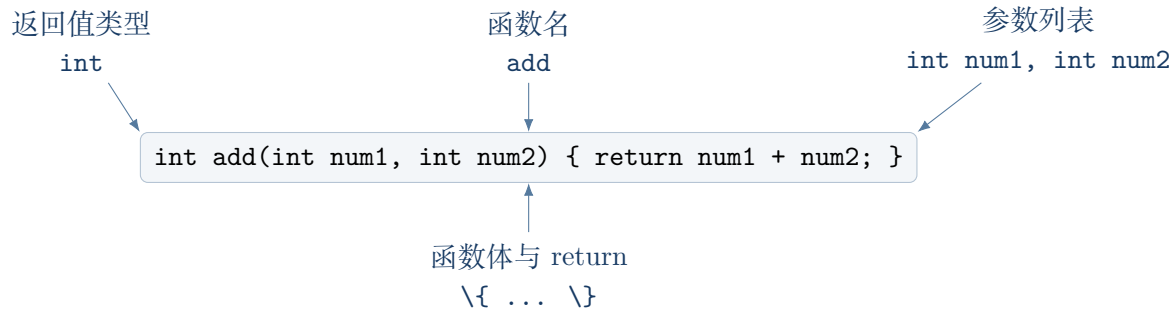
输出是 8。这里函数 add 负责“把两个整数相加”这一件事；main 只负责准备数据、调用函数并使用结果。

8 核心概念二：函数定义的结构

定义

一个典型函数定义可以写成：

```
return_type function_name(parameter_list) {
    statements;
    return expression;  // when a value is returned
}
```



8.1 五个要素逐项解释

要素	含义
返回值类型	说明函数返回给调用者的结果类型；若没有普通返回值，常用 void。
函数名	这个功能的名字，建议体现动作或目的，如 add、readSensor。
参数列表	说明函数需要哪些输入以及它们的类型；可以为空。
函数体	大括号内部的实际执行逻辑。
return 表达式	在有返回值函数中，把计算结果返回并结束本次函数执行。

9 核心概念三：形参与实参

定义

形参（parameter）写在函数定义/声明的参数列表中，是函数内部用来接收输入的局部参数。
实参（argument）写在调用表达式中，是调用时真正传入的值或表达式。

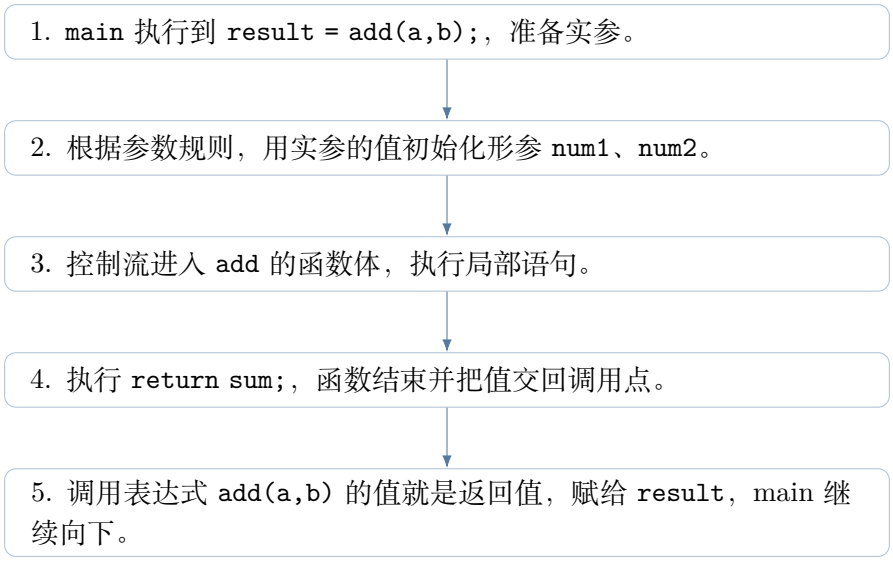
例子

```
int add(int x, int y) {   // x, y: parameters
    return x + y;
}

int a = 2;
int b = 7;
int c = add(a, b);        // a, b: arguments
```

调用发生时，可以先用一个非常朴素的模型理解：x 得到 a 的值 2，y 得到 b 的值 7，然后函数内部只操作自己的参数对象。

10 核心概念四：函数调用与返回过程



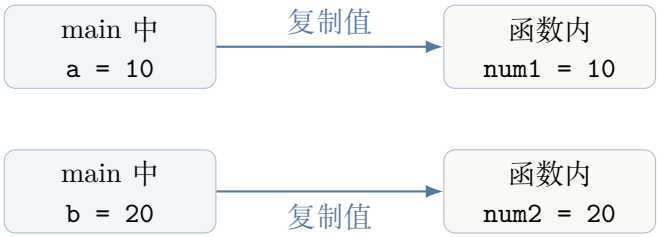
教学补全：return 不只是“给结果”

在有返回值函数中，return expression; 同时做两件事：**结束当前这次函数执行**，并**把表达式结果交给调用者**。在 void 函数中，可以写 return; 提前结束，但不能返回一个普通的非 void 值。

11 核心概念五：值传递为什么不会改掉实参

定义

当参数按值传递时，调用函数会用实参的值初始化形参。形参是函数内部的局部对象；对形参的修改通常不会直接修改外部实参。



交换 num1 与 num2，只是在右侧两个局部对象之间交换；左侧 a,b 仍保持原值。

```cpp
#include <iostream>

void swapValue(int num1, int num2) {
    int temp = num1;
    num1 = num2;
    num2 = temp;
    std::cout << num1 << ' ' << num2 << '\n';
}

int main() {
    int a = 10;
    int b = 20;
    swapValue(a, b);
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

函数内部先输出 20 10，回到 main 后输出仍是 10 20。

易错 / 纠错

“形参一开始和实参的值相同”不等于“形参就是实参”。值传递的核心是**复制值**。以后学到引用参数时，才会系统讨论怎样让函数直接作用到外部对象；本专题先把值传递彻底理解清楚。

12 核心概念六：四种基础函数形式

把“有没有参数”和“有没有返回值”做一个 2×2 组合，可以得到你笔记里的四种形式：

	无返回值	有返回值
无参数	`void f()`	`int f()`
有参数	`void f(int x)`	`int f(int x)`

```cpp
#include <iostream>

void test1() {
    std::cout << "no args, no return\n";
}

void test2(int a) {
    std::cout << a << '\n';
}

int test3() {
    return 42;
}

int test4(int a, int b) {
    return a + b;
}
```

总结

不要把这四种形式背成四个孤立模板。只问两个问题即可：**函数需不需要外部输入？调用者需不需要拿到一个结果？**这两个问题的答案决定“有没有参数”和“有没有返回值”。

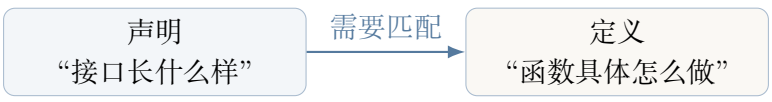
13 核心概念七：函数声明与函数定义

定义

函数声明告诉编译器“存在这样一个函数，它叫什么、返回什么类型、需要什么参数”。
函数定义在声明信息的基础上再给出函数体，真正提供实现。

```cpp
int max2(int a, int b);   // declaration

int max2(int a, int b) {  // definition
    return (a > b) ? a : b;
}
```

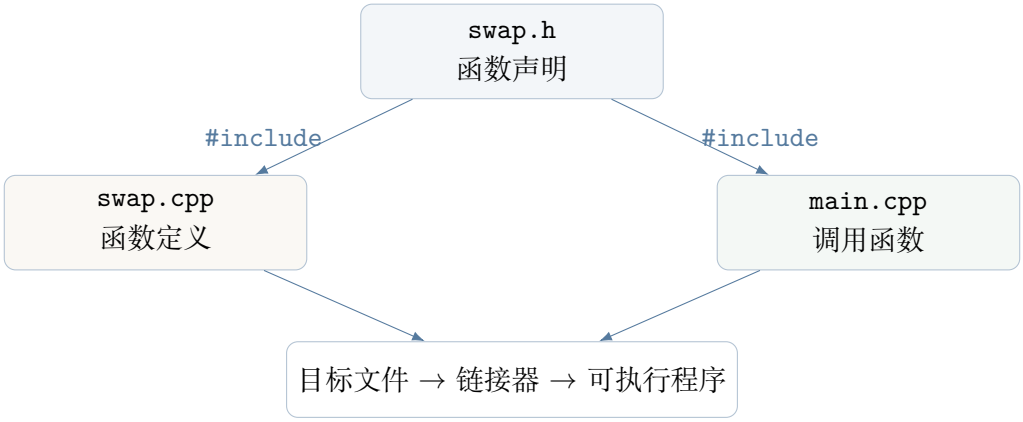


直观理解

编译器在看到调用时，至少要先知道这个名字代表一个什么函数，所以需要声明；而最终生成程序时，链接器还必须找到真正的函数代码，所以需要定义。

14 核心概念八：分文件编写到底发生了什么

14.1 最推荐的初学者三文件结构



swap.h

```
#pragma once
void swapValue(int num1, int num2);
```

swap.cpp

```
#include <iostream>
#include "swap.h"

void swapValue(int num1, int num2) {
    int temp = num1;
    num1 = num2;
    num2 = temp;
    std::cout << num1 << ' ' << num2 << '\n';
}
```

main.cpp

```
#include <iostream>
#include "swap.h"

int main() {
    int a = 10;
    int b = 20;
    swapValue(a, b);
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

14.2 为什么会出现 undefined reference

从“编译通过”到“链接失败”

如果 main.cpp 包含了 swap.h，编译器已经知道 swapValue 的声明，所以 main.cpp 可以被编译成目标文件；但如果构建命令只写：

```
g++ main.cpp -o app
```

那么链接器并没有得到 swap.cpp 生成的目标代码，就会找不到函数定义。正确的一步式写法例如：

```
g++ main.cpp swap.cpp -o app
```

也可以先分别使用 -c 编译成.o，再把这些.o 链接起来。



资料核验：Microsoft Learn 说明每个.cpp 翻译单元通常独立编译成目标文件，之后由链接器合并；GCC 官方文档也将预处理、编译、汇编、链接列为构建阶段，并说明 -c 会停止在目标文件阶段。

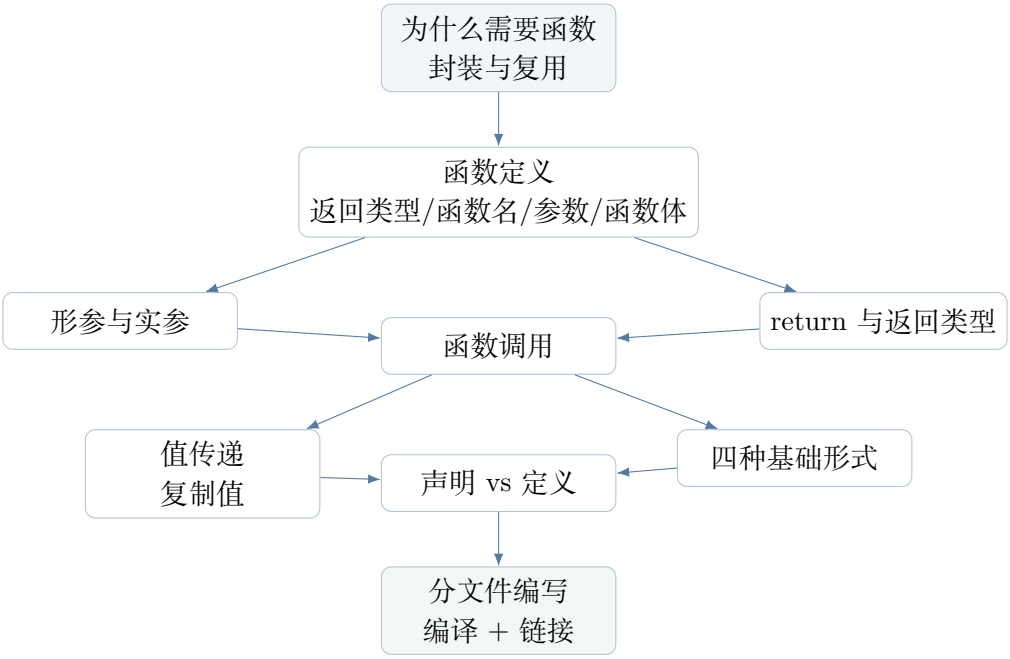
15 相近概念辨析

比较	概念 A	概念 B
形参 vs 实参	形参：写在函数接口中，是函数内部接收数据的局部对象	实参：写在调用处，是实际传入的值/表达式
声明 vs 定义	声明：告诉编译器“它长什么样”	定义：给出函数体“它怎么做”
void vs int	void：不返回普通结果值	int：调用者会获得一个 int 结果
值传递 vs 直接修改实参	值传递：复制值，形参变化不直接改实参	后续引用/指针参数才可建立对外部对象的直接作用（本专题不展开）
编译错误 vs 链接错误	编译错误：单个翻译单元在语法/类型等方面无法通过	链接错误：各目标文件无法正确拼接，例如声明有、定义缺失

16 高频易错点

1. 把形参和实参当成同一个变量，导致误判值传递效果。
2. void 函数仍写 return 某个 int;。
3. 声明后忘记写定义，或定义所在.cpp 没加入构建。
4. 声明和定义的参数类型不一致，形成“看起来同名、实际不是同一个函数”的问题。
5. 在头文件里随意放普通非 inline 函数定义，后续可能触发多重定义问题。
6. 误以为 #include "x.h" 会自动把 x.cpp 一起编译；事实上头文件提供声明，源文件是否参与构建由工程/编译命令决定。
7. 使用 using namespace std; 时自定义名为 swap、max 等函数容易与标准库名称混淆。初学示例可用 swapValue、max2 避免不必要干扰。

17 本专题知识地图



18 学完后的自检

不用看答案，先口头回答

1. 什么是函数？它和“把代码复制粘贴三遍”相比好在哪里？
2. 写出函数定义的基本结构，并说明每一部分含义。
3. 什么是形参，什么是实参？
4. 为什么 `void change(int x)` 不能直接把 main 里的 `a` 改掉？
5. `return` 在有返回值函数中完成哪两件事？
6. 四种基础函数形式是如何由两个“有/无”问题组合出来的？
7. 声明与定义各自解决什么问题？
8. `#include "swap.h"` 为什么不等于“自动编译 swap.cpp”？
9. 什么情况下更像编译错误，什么情况下更像链接错误？

19 知识点—题目覆盖表

核心知识点	基础题	提升题	跨学科
函数作用与封装	1,21	–	37
定义结构、返回类型、return	2,4,5,11,13,16,23,24	29	–
形参与实参	3,17	26	36,38
值传递	6,12,20,22	26	37
函数调用与嵌套调用	7	28,31,32	36,38
声明 vs 定义	8,11,23	27,30	37
分文件编写、编译与链接	9,10,14,19,25	27,30,33,35	–
四种基础函数形式	13,15,24	34	–

20 A. 基础巩固训练：25 题

20.1 A1. 基础单项选择题（1–10）

1. 函数最核心的教学用途之一是（）。

A. 让所有变量都变成全局变量

B. 把重复功能封装为可复用代码块

C. 让程序不再需要 main 函数

D. 让编译器跳过语法检查

2. 下面哪一项不是一个普通函数定义的基本组成部分？（）。

A. 返回值类型

B. 函数名

C. 参数列表

D. 必须存在的输入语句 cin

3. 在调用 int add(int x, int y) 时，表达式 add(a,b) 中的 a、b 称为（）。

A. 形参

B. 实参

C. 返回类型

D. 局部函数

4. 对于 int add(int a,int b) return a+b; ， return a+b; 的直接作用是（）。

A. 结束整个程序

B. 把表达式结果返回给调用者并结束本次函数执行

C. 重新声明 a、b

D. 只打印结果

5. 一个不需要向调用者返回结果值的函数，最常用的返回类型是 ()。

A. int

B. double

C. void

D. bool

6. 已知 void change(int x) x=100; ， main 中 int a=5; change(a); 之后 a 的值为 ()。

A. 5

B. 100

C. 未定义

D. 编译错误

7. 下面哪个写法是函数调用？()。

A. int add(int,int);

B. int add(int a,int b)return a+b;

C. add(2,3)

D. return add;

8. 下面哪个写法是函数声明而不是函数定义？()。

A. int max2(int a,int b);

B. int max2(int a,int b)return a>b?a:b;

C. max2(3,4);

D. return 0;

9. 在传统的.h + .cpp 分文件组织中，头文件最适合主要放置 ()。

A. 普通非 inline 函数的重复定义

B. 函数声明

C. main 的多份定义

D. 所有对象文件

10. 若 main.cpp 中能通过编译，但链接时出现 undefined reference to ‘swapValue(int,int)‘，最常见的原因是 ()。

A. 少写了 using namespace std;

B. 链接阶段没有找到该函数的定义

C. cout 不能输出整数

D. 函数参数必须写成 double

20.2 A2. 基础多项选择题（11–15）

11. 关于函数声明，下列说法正确的是（多选）。

A. 声明告诉编译器函数名及其类型信息

B. 声明一般没有函数体

C. 同一个兼容声明可以在程序中出现多次

D. 只要有声明，就绝不需要定义

12. 关于值传递，下列说法正确的是（多选）。

A. 形参由实参的值进行初始化

B. 修改形参通常不会直接修改实参

C. 形参与实参一定是同一个内存对象

D. 值传递适合让函数读取输入并在局部计算

13. 下列哪些函数定义在语法和返回规则上是合理的？（多选）。

A. void f1() return;

B. int f2() return 3;

C. void f3() return 3;

D. int f4(int x) return x*x;

14. 关于头文件与源文件，下列说法正确的是（多选）。

A. main.cpp 可以通过 `#include` 看到函数声明

B. 函数定义可以写在另一个.cpp 中

C. 每个.cpp 通常独立编译成目标文件后再链接

D. 只编译 main.cpp 就一定能自动找到任意其他.cpp 的函数定义

15. 关于函数的四种基础形式，下列说法正确的是（多选）。

A. 无参无返：void f()

B. 有参无返：void f(int x)

C. 无参有返：int f()

D. 有参有返：int f(int x)

20.3 A3. 基础填空题（16–20）

16. 函数定义 `int add(int a,int b)` 中，若要返回两数之和，函数体中的关键语句应填：_____。

17. 在 int add(int x,int y) 中，x、y 称为_____；在 add(a,b) 中，a、b 称为_____。。

18. C++ 中表示“函数不返回普通结果值”的关键字是_____。。

19. 把函数声明集中写在文件 swap.h 中，并在 main.cpp 中使用时，通常需要写预处理指令_____。。

20. 执行 int a=10; void f(int x)x=20; f(a); 后，a 的值是_____。。

20.4 A4. 基础解答题（21–25）

21. 写出一个函数 square，接收一个 int 参数并返回它的平方，然后给出一次调用。

22. 说明为什么 valueSwap(int a,int b) 在函数内部交换 a、b 后，main 中的两个变量仍不变。

23. 用一句规范的话区分函数声明与函数定义。

24. 指出并改正：void f() int x=3; return x; 。

25. 给出最简单的三文件组织：calc.h、calc.cpp、main.cpp，并说明 g++ 链接时要包含哪些.cpp。

21 B. 中等偏下提升训练：10 题

21.1 B1. 单项选择题（26–28）

26. 阅读代码：int inc(int x)x++; return x; int a=4; int b=inc(a); 执行后 a、b 分别为 ()。

A. 4,5
B. 5,5
C. 4,4
D. 5,4

27. 下面哪一组最能解释“声明正确、定义也存在，但仍可能出现链接问题”？()。

A. 定义所在的.cpp 没有被加入构建/链接
B. 函数里没有 cout
C. main 写在第一行
D. 参数名不同

28. 若 int max2(int a,int b) return a>b?a:b; ，则 max2(max2(3,7),5) 的结果为 ()。

A. 3
B. 5
C. 7

D. 15

21.2 B2. 多项选择题（29–30）

29. 下列代码片段哪些需要修改后才能作为标准的初学者示例？（多选）。

A. void f() return 1;

B. int g() return 1;

C. void h(int x) std::cout«x;

D. int k() int t=0; return t;

30. 出现 undefined reference / unresolved external symbol 时，下列哪些检查方向合理？（多选）。

A. 函数是否真正定义

B. 定义所在.cpp 是否加入构建

C. 声明与定义的参数类型是否一致

D. 是否把所有 int 都改成 double

21.3 B3. 填空题（31–33）

31. 若 int twice(int x)return 2*x;，则 twice(twice(3)) = _____。

32. 若 int bigger(int a,int b)return a>b?a:b;，则 bigger(4,9) 返回 _____。

33. 有 main.cpp 和 tools.cpp 两个实现文件，使用 g++ 一步完成编译与链接的一条命令可写为 _____。

21.4 B4. 综合大题（34–35）

34. 函数形式与返回规则综合。阅读下面四个函数：

```
void f1() { }
void f2(int x) { std::cout << x; }
void f3(int a, int b) { return a + b; }
void f4() { int t = 5; return t; }
```

（1）分别判断它们属于哪一种“有/无参数、有/无返回值”的基础形式；（2）指出不符合返回规则的函数；（3）在尽量少改代码的前提下修正，并说明理由。

35. 分文件与链接综合。现有三个文件：

```
// calc.h
#pragma once
int add(int a, int b);
```

```cpp
// calc.cpp
#include "calc.h"
int add(int a, int b) { return a + b; }
```

```cpp
// main.cpp
#include <iostream>
#include "calc.h"
int main() { std::cout << add(2, 3); }
```

(1) 说明每个文件的职责；(2) 为什么 `g++ main.cpp -o app` 可能链接失败？(3) 给出正确的一步式构建命令；(4) 若声明误写为 `int add(double,double);`，会出现什么层面的问题？

22 C. 跨学科迁移训练：3 题

36. 跨学科单选（程序设计 × 传感器）。某传感器把 0 1023 的 ADC 数字量线性换算为 0 5 V。已给函数 double voltage(int adc) return 5.0*adc/1023; 。若只讨论函数知识，下列最准确的是 ()。

A. adc 是实参

B. 调用 voltage(512) 时，512 是实参，adc 是形参

C. 函数没有返回值

D. 函数一定修改调用者中的变量

37. 跨学科多选（程序设计 × 机器人控制）。机器人程序中用 int clampSpeed(int v) 把速度限制在安全范围，主程序多次调用它。关于这种做法，下列说法正确的是（多选）。

A. 把重复的限制逻辑封装成函数能减少重复代码

B. 函数名可以表达该代码块的功能

C. 若采用值传递，函数内部改变形参 v 不等于直接改变主程序原变量

D. 只要写了函数声明，完全不需要函数定义

38. 跨学科填空（程序设计 × 轮速测量）。轮速传感器每转输出 20 个脉冲。题干已给关系：转数 = 脉冲数 / 20。若写 double turns(int pulses) return pulses/20.0; ，调用 turns(50) 的返回值为 _____ 转。

23 全部答案速查

基础单选： 1.B 2.D 3.B 4.B 5.C 6.A 7.C 8.A 9.B 10.B
基础多选： 11.ABC 12.ABD 13.ABD 14.ABC 15.ABCD
基础填空： 16. `return a+b;`；17. 形参、实参；18. `void`；19. `#include "swap.h"`；20. 10
基础解答： 21–25 见详细解析。

提升单选： 26.A 27.A 28.C
提升多选： 29.A 30.ABC
提升填空： 31.12 32.9 33. `g++ main.cpp tools.cpp -o app`
提升综合： 34–35 见详细解析。

跨学科： 36.B 37.ABC 38.2.5 转

24 基础题详细解析

24.1 单项选择 1–10

1. 正确答案：B 难度：基础 考查：基础概念
函数把一段具有明确功能的代码封装起来，可被多次调用，从而减少重复并增强结构性。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

2. 正确答案：D 难度：基础 考查：基础概念
函数定义通常由返回类型、函数名、参数列表和函数体构成；是否使用 cin 与函数本身无关。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

3. 正确答案：B 难度：基础 考查：基础概念
定义中的 x、y 是形参；调用处传进去的 a、b 是实参。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

4. 正确答案：B 难度：基础 考查：基础概念
return 在有返回值函数中结束当前函数并把值交回调用处。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

5. 正确答案：C 难度：基础 考查：基础概念
void 表示该函数不向调用者返回普通结果值。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

6. 正确答案：A 难度：基础 考查：基础概念
值传递时形参 x 是由 a 的值初始化出的局部副本，修改 x 不改变 a。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

7. 正确答案：C 难度：基础 考查：基础概念
add(2,3) 才是调用表达式；A 是声明，B 是定义。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

8. 正确答案：A 难度：基础 考查：基础概念
以分号结束且没有函数体的是声明。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

9. 正确答案：B 难度：基础 考查：基础概念
头文件通常放声明，由需要的.cpp 通过 `#include` 获知函数接口。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

10. 正确答案：B 难度：基础 考查：基础概念
编译器只需看到声明即可生成调用；链接器还必须找到对应定义的目标代码。其余选项的问题在于：它们要么把函数与输入/输出语句混为一谈，要么把“声明、定义、调用”三个不同阶段混在一起。做这类题先问“题目描述的是接口、实现，还是一次实际调用”。

24.2 多项选择 11–15

11. 正确答案：ABC 难度：基础
A 正确：声明提供接口信息。B 正确：典型函数声明以分号结束。C 正确：兼容声明可重复出现。D 错误：函数若被实际调用，通常还需要可供链接器找到的定义。
一句话总结：多选题不要凭“感觉像对”，必须逐项对照定义、返回规则或编译/链接流程。

12. 正确答案：ABD 难度：基础
A 正确。B 正确。C 错误：值传递下形参与实参是不同对象。D 正确：函数可用副本进行局部计算。
一句话总结：多选题不要凭“感觉像对”，必须逐项对照定义、返回规则或编译/链接流程。

13. 正确答案：ABD 难度：基础
A 合理：void 函数可写 return;。B 合理。C 错误：void 函数不能返回 int 表达式。D 合理。
一句话总结：多选题不要凭“感觉像对”，必须逐项对照定义、返回规则或编译/链接流程。

14. 正确答案：ABC 难度：基础
A 正确。B 正确。C 正确。D 错误：构建命令或工程必须把相应源文件纳入编译和链接。
一句话总结：多选题不要凭“感觉像对”，必须逐项对照定义、返回规则或编译/链接流程。

15. 正确答案：ABCD 难度：基础
A、B、C、D 都是常见的四种基础组合。
一句话总结：多选题不要凭“感觉像对”，必须逐项对照定义、返回规则或编译/链接流程。

24.3 填空题 16–20

16. 标准答案： return a + b; 难度：基础
知识点：函数接口与值传递。 **解析：**有返回值函数要把结果通过 return 交还调用处。

17. 标准答案：形参；实参 难度：基础
知识点：函数接口与值传递。 **解析：**定义处是形式参数，调用处是实际参数。

18. 标准答案： void 难度：基础
知识点：函数接口与值传递。 **解析：**void 是无返回值函数最常用的返回类型。

19. 标准答案： `#include "swap.h"` 难度：基础
知识点：函数接口与值传递。 **解析：**`#include` 使当前翻译单元在预处理阶段获得该声明。

20. 标准答案： 10 难度：基础
知识点：函数接口与值传递。 **解析：**x 是 a 的值副本，修改 x 不会改变 a。

24.4 基础解答题 21–25

21. 题意分析：写出一个函数 square，接收一个 int 参数并返回它的平方，然后给出一次调用。
标准作答：

```
int square(int x) {
    return x * x;
}
// call:
int y = square(6);
```

为什么：返回类型是 int，函数名是 square，形参 x 接收实参 6 的值，结果 36 通过 return 返回。
易错点：不要只写最终代码而不说明返回类型、参数与调用之间的关系。

22. 题意分析：说明为什么 valueSwap(int a,int b) 在函数内部交换 a、b 后，main 中的两个变量仍不变。
标准作答：

因为普通 int 形参采用值传递。调用时只是把实参的值复制给形参；函数内部交换的是两个局部副本，实参本身未被修改。
为什么：关键是区分“值相同”和“对象相同”。值传递只复制值，不让形参与实参成为同一个对象。
易错点：不要只写最终代码而不说明返回类型、参数与调用之间的关系。

23. 题意分析：用一句规范的话区分函数声明与函数定义。
标准作答：
函数声明只说明函数的名字与类型接口；函数定义在此基础上给出函数体，真正提供可执行实现。
为什么：编译某个调用点时需要先知道声明；链接时还需要找到相应定义。
易错点：不要只写最终代码而不说明返回类型、参数与调用之间的关系。

24. 题意分析：指出并改正：void f() int x=3; return x; 。
标准作答：
错误在于函数返回类型为 void，却返回了 int 表达式。若需要返回 3，应改为

```
int f() {
    int x = 3;
    return x;
}
```

若不需要返回值，则保持 void 并使用 return; 或直接执行到函数体末尾。
为什么：返回类型和 return 表达式必须匹配。
易错点：不要只写最终代码而不说明返回类型、参数与调用之间的关系。

25. 题意分析：给出最简单的三文件组织：calc.h、calc.cpp、main.cpp，并说明 g++ 链接时要包含哪些.cpp。
标准作答：
calc.h

```
#pragma once
int add(int a, int b);
```

calc.cpp

```
#include "calc.h"
int add(int a, int b) { return a + b; }
```

main.cpp

```
#include <iostream>
#include "calc.h"
int main() { std::cout << add(2,3); }
```

构建：`g++ main.cpp calc.cpp -o app`。
为什么：若只把 main.cpp 交给 g++，链接器就没有 calc.cpp 生成的目标代码可供合并。
易错点：不要只写最终代码而不说明返回类型、参数与调用之间的关系。

25 提升题详细解析

25.1 单项选择 26–28

26. 正确答案：A 难度：中等偏下
思路：先沿调用顺序追踪局部形参和返回值，再判断外部变量是否被直接修改。
x 是 a 的副本，因此 x++ 不改变 a；return x 把 5 返回给 b。

27. 正确答案：A 难度：中等偏下
思路：先沿调用顺序追踪局部形参和返回值，再判断外部变量是否被直接修改。
跨文件编译时，每个需要参与最终程序的实现文件都必须产生目标文件并交给链接器。参数名不同不会改变函数类型。

28. 正确答案：C 难度：中等偏下
思路：先沿调用顺序追踪局部形参和返回值，再判断外部变量是否被直接修改。
先算内层 max2(3,7)=7，再算 max2(7,5)=7。

25.2 多项选择 29–30

29. 正确答案：A 难度：中等偏下
A 错：void 不能返回 int 表达式。B、C、D 都符合基础语义。
易错点：把“语法能否通过”和“最终能否链接”混成一个问题。

30. 正确答案：ABC 难度：中等偏下
A、B、C 都直接关系到符号能否被链接器解析。D 没有普遍依据。
易错点：把“语法能否通过”和“最终能否链接”混成一个问题。

25.3 填空题 31–33

31. 标准答案： 12 难度：中等偏下
内层 twice(3)=6，外层 twice(6)=12。

32. 标准答案： 9 难度：中等偏下
条件 4>9 为假，因此条件运算符选择 b。

33. 标准答案： g++ main.cpp tools.cpp -o app 难度：中等偏下
两个源文件都必须作为输入，最终才能把相应目标代码链接进 app。

25.4 综合大题 34

1. 题目到底在问什么 要把“是否有参数”和“是否返回值”两个维度分开，再检查返回类型与 return 是否匹配。
2. 判断四种形式 `f1`: 无参无返；`f2`: 有参无返；`f3`: 有参无返（按函数头看）；`f4`: 无参无返（按函数头看）。
3. 错误定位 `f3`、`f4` 的函数头都写成 `void`，却返回了 int 表达式。
4. 最小修改 若保留它们“确实要返回整数”的意图，应改为：

```
int f3(int a, int b) { return a + b; }
int f4() { int t = 5; return t; }
```

修正后，`f3` 变成“有参有返”，`f4` 变成“无参有返”。
5. 本题最容易卡在哪里 不要只看函数体里有没有 `return` 来决定“有返回值”；真正决定函数接口的是函数头的返回类型，二者必须一致。

25.5 综合大题 35

1. 文件职责 `calc.h` 提供声明；`calc.cpp` 提供定义；`main.cpp` 获得声明后进行调用。
2. 为什么只编译 main.cpp 会失败 编译 main.cpp 时，头文件中的声明足够让编译器接受 `add(2,3)`；但链接阶段没有 `calc.cpp` 的目标代码，链接器找不到 `add(int,int)` 的实现。
3. 正确命令 `g++ main.cpp calc.cpp -o app`。
4. 若声明写成 `int add(double,double);` 调用点会按 double 参数版本生成对相应符号的引用，而 calc.cpp 实际定义的是 `int add(int,int)`。它们是不同的函数类型；在分开的翻译单元中，这通常最终表现为链接器找不到被调用的那个签名。
5. 合理性检查 这解释了为什么“函数名看起来一样”仍可能链接失败：C++ 的函数符号与参数类型有关。

26 跨学科迁移题详细解析

36. 正确答案：B
当前专题知识： 形参与实参。
借用的外部情境： ADC 数字量到电压的线性换算。
为什么不算超纲： 换算公式已经完整写在函数体中，不要求预先学习模电或 ADC 原理。
迁移点： 无论输入代表“普通整数”还是“传感器采样值”，定义处的 `adc` 都是形参，调用处的 512 都是实参。

37. 正确答案：ABC
当前专题知识： 函数封装、函数名表达功能、值传递、声明与定义。
借用的外部情境： 机器人速度限制。
为什么不算超纲： 题目不要求知道机器人动力学，只说明这是一个会被重复执行的限制逻辑。
迁移点： 现实工程程序中的“重复操作”同样可以被封装成函数；值传递和声明/定义规则并不会因为应用背景改变。

38. 标准答案：2.5 转
当前专题知识：函数调用、形实参对应、return。
借用的外部情境：轮速传感器脉冲计数。
为什么不算超纲：题干已经给出“转数 = 脉冲数 / 20”。
计算：50/20.0 = 2.5，所以 `turns(50)` 的调用表达式值为 2.5。

27 基础解题步骤总结

遇到函数题，可以按下面顺序检查：

1. **先看函数头：**返回类型是什么？参数有几个？各是什么类型？
2. **再看调用处：**实参是什么？传给哪个形参？
3. **判断传递方式：**本专题默认普通值参数，形参是副本。
4. **顺着函数体执行：**局部变量怎样变化？何时 return？
5. **回到调用点：**返回值被赋给谁，或被怎样使用？
6. **跨文件时额外检查：**调用处看见声明了吗？定义存在吗？定义所在.cpp 参与构建了吗？

28 后续课程连接

后续主题	与本专题的连接
引用与指针	解释函数怎样不仅“读取副本”，还可以直接作用到调用者对象。
数组与字符串	常把数组处理逻辑封装进函数，需要继续学习数组参数与指针退化等规则。
函数重载	同名函数可以根据参数类型/数量形成不同接口。
类与对象	成员函数把“函数”与“对象状态”结合，是面向对象程序设计的核心。
数据结构与算法	每个算法操作几乎都会被组织成函数；递归、复杂度分析也以函数为载体。
嵌入式/机器人	读取传感器、滤波、控制、通信、状态机常按函数或模块分层组织。

29 参考资料与权威核验

参考文献

[1] Microsoft Learn, *Functions (C++)*. 重点核验：函数定义、声明、参数、返回类型。
https://learn.microsoft.com/en-us/cpp/cpp/functions-cpp?view=msvc-170

[2] cppreference, *Functions / Function declaration*. 重点核验：函数声明与定义的语言语义。
https://en.cppreference.com/w/cpp/language/functions

[3] cppreference, *return statement*. 重点核验：`void` 与有返回值函数的 return 规则。
https://en.cppreference.com/w/cpp/language/return

[4] Microsoft Learn, *Header files (C++)*. 重点核验：头文件声明、各.cpp 独立编译、链接阶段合并。
https://learn.microsoft.com/en-us/cpp/cpp/header-files-cpp?view=msvc-170

[5] GNU GCC Manual, *Overall Options*. 重点核验：预处理、编译、汇编、链接以及 `-c` 的含义。
https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html

[6] Bjarne Stroustrup, *FAQ / A History of C++: 1979–1991*. 重点核验：C++ 的早期时间线。
https://stroustrup.com/bs_faq.html

本专题最后一句话

函数不是“又一种语法格式”，而是程序开始具有结构的地方：**定义功能边界、接收输入、完成局部计算、返回结果，再通过声明和链接把不同文件连接起来**。一旦把这条链真正理解，后面的引用、数组、类、数据结构都会顺畅很多。